

PROJECT REPORT: DSA/CODING MENTOR AGENT

Autonomous Dual-Agent AI Framework

Type: Internship Capstone

Release Date: July 11, 2026

Tech: LangChain & Streamlit

Overview: An autonomous multi-agent system designed to guide students through Data Structures and Algorithms (DSA) challenges using the **Socratic Method**. Instead of directly delivering code answers, the platform systematically prompts students with tailored hints and conceptual questions to mirror an experienced human instructor.

KEY PERFORMANCE & DESIGN METRICS

40+

DSA
PATTERNS

2

POLYGLOT
LANGS

5s

TIMEOUT
LIMIT

10

VALIDATION
CASES

0

DIRECT
CODE LEAKS

13

MVP DEV
DAYS

DUAL-AGENT ARCHITECTURE

[1] Research Agent — "The Algorithmic Architect"

Responsible for decomposing and mapping untagged, story-based technical challenges to fundamental algorithm groups.

- Extracts structural patterns (e.g., "Connecting cities minimal cost" → Minimum Spanning Tree).
- Generates a structural **Strategy Brief JSON** containing target space/time constraints, failure edge conditions, and structured dynamic hints.

[2] Action Agent — "The Polyglot Coach"

Manages live dialogue interaction loops with student users via the interactive interface panels.

- Enforces rigorous Socratic hint progressions to steer reasoning without spoiling code logic.
- Operates the secure cross-language execution backend sandbox for active Python and Java validation.

PROJECT DIRECTORY STRUCTURE

```
dsa-mentor-agent/  
├── main.py           # Entry point – launches Streamlit app  
├── requirements.txt  # Python dependencies  
├── .env             # API configuration parameters  
└── README.md
```

```

├─ agents/
│   ├── research_agent.py # Pattern extraction & Strategy Brief JSON engine
│   ├── action_agent.py   # Socratic interaction handler
│   └─ orchestrator.py    # Agent handoff & system control loop
├─ knowledge/
│   ├── patterns.json      # 40+ structured paradigms with granular complexity
│   └─ pattern_matcher.py # Keyword → pattern parsing logic
├─ executor/
│   ├── python_runner.py  # Isolated subprocess execution + timeout monitors
│   ├── java_runner.py    # javac compilation + runtime telemetry tracking
│   └─ sandbox.py         # Temporary system workspace & dynamic network blocker
├─ prompts/
│   ├── research_prompt.txt # Algorithmic core identification instructions
│   └─ mentor_prompt.txt   # Socratic persona & output compliance filters
├─ ui/
│   ├── chat_panel.py     # Left interface layout: Conversation stream
│   └─ code_panel.py      # Right interface layout: Code editor & console execution
└─ tests/
    ├── test_agents.py    # Agent state flow verification tests
    └─ test_executor.py   # Code sandbox & malicious code termination verification

```

DEVELOPMENT ROADMAP & TIMELINE

Phase / Date	Milestone Target	Detailed Implementation Scope
Phase 1: Core Logic & Intelligence		
Day 1 (Jun 28)	Architecture Config	Establish prompt boundaries, environment matrices, and multi-agent roles.
Day 2 (Jun 29)	Data Contract Mapping	Formalize strict JSON interface models passing parameters between agents.
Day 3 (Jun 30)	Mock Pipeline Evaluation	Simulate full execution path loops without consuming remote LLM live API keys.
Phase 2: Pattern Knowledge Integration		
Day 4 (Jul 1)	Knowledge Base Mapping	Populate <code>patterns.json</code> with deep metadata across 40+ paradigm targets.
Day 5 (Jul 2)	Keyword Parsing Logic	Program fast algorithmic structural discovery parsing heuristic tokens.
Day 6 (Jul 3)	API Wrapping Models	Build the pipeline handling dynamic generation of structural Strategy Briefs.
Phase 3: Polyglot Code Execution Engine		
Day 4 (Jul 4)	Python Subprocess Sandboxing	Configure isolated Python processes interacting safely through disk buffers.
Day 8 (Jul 5)	Java Compilation Pipeline	Wire <code>javac</code> and underlying JVM runtimes to measure performance metrics.
Day 9 (Jul 6)	Timeout & Threat Defenses	Deploy a 5-second hard execution watchdog to terminate infinite logic blocks.
Phase 4: UI, Testing & Polish		
Day 10-11 (Jul 7-8)	Streamlit Panel Design	Build the responsive side-by-side Chat Panel and Interactive Code Panel UI.
Day 12 (Jul 9)	Validation Matrix Evaluation	Execute comprehensive integration tests profiling common student bugs.
Day 13 (Jul 10)	System Convergence Review	Run final end-to-end user validations prior to release sign-off.
July 11, 2026	Project Delivery	Official Capstone Submission to Evaluation Panel

ALGORITHMIC KNOWLEDGE MATRIX (40+ PARADIGMS)

Arrays, Strings & Sorting

- Sliding Window
- Two Pointers
- Fast & Slow Pointers
- Prefix Sum
- Binary Search
- Merge / Quick Sort
- Counting Sort

Graphs, Trees & Lists

- BFS & DFS Tracing
- Kruskal & Prim MST
- Topological Sort
- Dijkstra's Shortest Path
- Union-Find Structural Maps
- Tree Traversals & LCA
- Linked List Mutation

DP & Advanced Paradigms

- Memoization / Tabulation
- 0/1 & Unbounded Knapsack
- Longest Common Subsequence
- Backtracking Exploration
- Greedy Optimizations
- Monotonic Stacks / Queues
- Trie Structure Matching

SECURITY SAFEGUARDS & SOCRATIC GUARDRAILS

Defense / System Focus	Implementation Standard
Network Disabled	Execution wrappers completely isolated; socket requests explicitly blocked in runtime sandbox environments.
Hard Evaluation Timeouts	Strict 5-second thread lifecycle termination to block CPU-hogging infinite loop implementations.
Custom Output Parsing	Regex structural scanner filters raw LLM payloads, stripping out code snippets before UI injection until benchmarks are unlocked.
Progressive Hint Delivery	Logical scaffolding tracks conversation state. Hints are revealed contextually one element at a time based on active code parsing.

SYSTEM SETUP & LAUNCH DIRECTIVES

1. Clone Remote Workspace:

```
git clone https://github.com/your-username/dsa-mentor-agent.git
cd dsa-mentor-agent
```

2. Provision Dependencies: (Requires Python 3.10+ and Java JDK 17+ environment installations)

```
pip install -r requirements.txt
```

3. Inject API Authorization Framework: Create a local `.env` template file:

```
LLM_API_KEY=your_secured_credential_token_here
LLM_PROVIDER=gemini
```

4. Boot Interface Server: Run using the local host port multiplexer:

```
streamlit run main.py
```

The system maps instantly to the browser endpoint: <http://localhost:8501>